

1.1 PROJECT TITLE

Snort 3 Based Intrusion Detection and Network Traffic Monitoring System

1.2 PROJECT OVERVIEW

- An Intrusion Detection System (IDS) is a security mechanism used to monitor network traffic and identify suspicious or potentially malicious activities.
- In this project, Snort 3 was installed and configured on an Ubuntu virtual machine and used as a network-based Intrusion Detection System. Kali Linux was used as the testing machine to generate network traffic against the Ubuntu machine.
- Custom Snort rules were created to detect activities such as ICMP traffic and network scanning attempts. The generated traffic was monitored using Snort alerts and correlated with packet captures in Wireshark.
- The project provided practical experience in Snort configuration, rule creation, network traffic analysis, IDS alert monitoring, troubleshooting, and security event investigation

1.3 OBJECTIVES

The main objectives of this project were:

- To install and configure Snort 3.
- To understand the basic architecture of Snort.
- To configure Snort for network traffic monitoring.
- To create custom Snort detection rules.
- To detect ICMP traffic.
- To perform controlled network scanning from Kali Linux.
- To observe Snort alerts.
- To capture and analyze network traffic using Wireshark.
- To correlate Snort alerts with captured packets.

- To troubleshoot network and Snort configuration issues.
- To understand the practical working of a Network IDS.

1.4 LAB ENVIRONMENT & TOOLS USED

Components	Details
Monitoring Machine	Ubuntu
Testing Machine	Kali Linux
Virtualization	Oracle VirtualBox
IDS	Snort 3
Packet Analyzer	Wireshark
Network Type	Virtual/Host-only laboratory network
Detection Method	Rule based network traffic network

1.5 NETWORK ARCHITECTURE

Kali Linux → Network Traffic → Ubuntu + Snort 3

Wireshark was used to capture and inspect the packets generated during testing.

1.5 TOOLS AND TECHNOLOGIES

5.1 Snort 3

- Snort 3 is an open-source network security tool capable of detecting suspicious network traffic using rules.
- It can be used for:
 - Intrusion detection
 - Network traffic inspection

- Rule-based detection
- Security alert generation
- Network monitoring

5.2 Kali Linux

Kali Linux was used as the testing machine. It provided networking and security tools that could generate controlled traffic for testing the IDS.

5.3 Ubuntu

Ubuntu was used as the Snort monitoring machine. Snort 3 was installed and configured on this system.

5.4 Wireshark

Wireshark was used to capture and analyse network packets. It helped verify whether the traffic that generated Snort alerts was actually present in the network capture.

5.5 VirtualBox

VirtualBox was used to create and connect the virtual machines required for the security laboratory.

1.6 SNORT 3 INSTALLATION

- The Snort 3 installation was performed on Ubuntu.
- Before installation, the system was updated and required dependencies and libraries were installed.
- The Snort source code was then configured and compiled.
- Typical build steps included:

`./bootstrap`

`./configure`

Make -j\$(nproc)

Sudo make install

```
-- Build files have been written to: /home/sakshi/Desktop/snort3/build
sakshi@sakshi-VirtualBox: ~/Desktop/snort3$ cd build make -j$(nproc)
bash: cd: too many arguments
sakshi@sakshi-VirtualBox: ~/Desktop/snort3$ cd build
sakshi@sakshi-VirtualBox: ~/Desktop/snort3/build$ make -j$(nproc)
[ 0%] Building CXX object src/connectors/unixdomain_connector/CMakeFiles/unixdomain_connector.dir/unixdomain_connector.cc.o
[ 0%] Building CXX object src/framework/CMakeFiles/framework.dir/codex.cc.o
[ 0%] Building CXX object src/framework/CMakeFiles/framework.dir/cursor.cc.o
[ 0%] Building CXX object src/connectors/unixdomain_connector/CMakeFiles/unixdomain_connector.dir/unixdomain_connector_module.cc.o
[ 0%] Building CXX object src/framework/CMakeFiles/framework.dir/data_bus.cc.o
[ 0%] Built target unixdomain_connector
[ 0%] Building CXX object src/actions/CMakeFiles/lps_actions.dir/actions_module.cc.o
[ 0%] Building CXX object src/actions/CMakeFiles/lps_actions.dir/lps_actions.cc.o
[ 0%] Building CXX object src/actions/CMakeFiles/lps_actions.dir/act_alert.cc.o
[ 0%] Building CXX object src/framework/CMakeFiles/framework.dir/inspector.cc.o
[ 0%] Building CXX object src/actions/CMakeFiles/lps_actions.dir/act_block.cc.o
[ 0%] Building CXX object src/framework/CMakeFiles/framework.dir/lps_action.cc.o
[ 0%] Building CXX object src/actions/CMakeFiles/lps_actions.dir/act_drop.cc.o
[ 0%] Building CXX object src/framework/CMakeFiles/framework.dir/lps_option.cc.o
[ 1%] Building CXX object src/actions/CMakeFiles/lps_actions.dir/act_file_id.cc.o
[ 1%] Building CXX object src/framework/CMakeFiles/framework.dir/parameter.cc.o
[ 1%] Building CXX object src/actions/CMakeFiles/lps_actions.dir/act_log.cc.o
[ 1%] Building CXX object src/framework/CMakeFiles/framework.dir/plg_pen.cc.o
[ 1%] Building CXX object src/actions/CMakeFiles/lps_actions.dir/act_pass.cc.o
[ 1%] Building CXX object src/framework/CMakeFiles/framework.dir/module.cc.o
[ 1%] Building CXX object src/actions/CMakeFiles/lps_actions.dir/act_read.cc.o
```

```
sakshi@sakshi-VirtualBox: ~/Desktop/snort3
sakshi@sakshi-VirtualBox: ~/Desktop/snort3$ cd libdaq
sakshi@sakshi-VirtualBox: ~/Desktop/libdaq$ ./bootstrap
+ autoreconf -ivf --warnings=all
autoreconf: export WARNINGS=all
autoreconf: Entering directory '.'
autoreconf: configure.ac: not using Gettext
autoreconf: running: aclocal --force -I m4
autoreconf: configure.ac: tracing
autoreconf: running: libtoolize --copy --force
libtoolize: putting auxiliary files in '.'.
libtoolize: copying file './ltmain.sh'
libtoolize: putting macros in AC_CONFIG_MACRO_DIRS, 'm4'.
libtoolize: copying file 'm4/libtool.m4'
libtoolize: copying file 'm4/ltoptions.m4'
libtoolize: copying file 'm4/ltugar.m4'
libtoolize: copying file 'm4/ltversion.m4'
libtoolize: copying file 'm4/lt-obsolete.m4'
autoreconf: configure.ac: not using Intltool
autoreconf: configure.ac: not using Gtkdoc
autoreconf: running: aclocal --force -I m4
autoreconf: running: /usr/bin/autoconf --force
```

After installation, the Snort version was checked to confirm that the installation was successful.

Snort -V

```
sakshi@sakshi-VirtualBox:~/Desktop/snort3/build$ sudo ldconfig
sakshi@sakshi-VirtualBox:~/Desktop/snort3/build$ snort -V

_*> Snort++ <*_
o" }-
'''
  Version 3.12.2.0
  By Martin Roesch & The Snort Team
  http://snort.org/contact#team
  Copyright (C) 2014-2026 Cisco and/or its affiliates. All rights reserved.
  Copyright (C) 1998-2013 Sourcefire, Inc., et al.
  Using DAQ version 3.0.27
  Using libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)
  Using LuaJIT version 2.1.1761786844
  Using LZMA version 5.8.3
  Using OpenSSL 3.5.5 27 Jan 2026
  Using PCRE2 version 10.46 2025-08-27
  Using ZLIB version 1.3.1
```

1.7 SNORT CONFIGURATION

- Snort 3 uses the Lua-based configuration system.
- The main configuration file used in the project was:

`/usr/local/etc/snort/snort.lua`

- The configuration was checked and modified according to the laboratory network environment.
- The Snort configuration was then validated using:

`Snort -c /usr/local/etc/snort/snort.lua -T`

- The -T option performs a configuration test.
- A successful validation confirmed that the Snort configuration and rules could be loaded correctly.

```

sakshi@sakshi-VirtualBox:~/desktop/snort3/build$ snort -c /usr/local/etc/snort/snort.lua -T
.....
o")>- Snort++ 3.12.2.0
.....
Loading /usr/local/etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  output
  ips
  binder
  wizard
  file_inspect
  stream
  stream_ip
  stream_icmp
  stream_tcp
  stream_user
  stream_file
  arp_spoof
  back_orifice
  dns
  lnag
  netflow
  normalizer

```

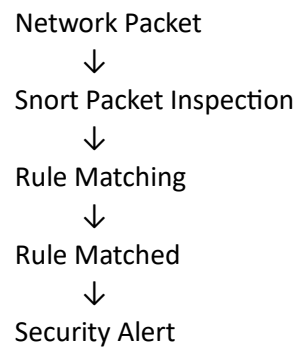
1.8 CUSTOM SNORT RULES

- Custom rules were created to detect specific network activities.
- The rules were stored in:
local.rules
- The rules were designed according to the laboratory network and the traffic being tested.
- A basic ICMP detection rule can be written as:
alert icmp any any -> any any
(msg:"ICMP Traffic Detected";
sid:1000001; rev:1;)
- This rule generates an alert when ICMP traffic is detected.
- Rule Components:

Component	Meaning
"alert"	Generate an alert
"icmp"	Monitor ICMP traffic
"any"	Any source port/address where applicable
"->"	Traffic direction
"any"	Any destination
"msg"	Alert message
"sid"	Unique Snort rule ID
"rev"	Rule revision

1.9 ICMP TRAFFIC DETECTION

- ICMP traffic was generated from Kali Linux toward the Ubuntu/Snort machine.
- Example:
Ping <Ubuntu-IP>
- The ping packets generated ICMP request and reply traffic.
- Snort inspected the packets and matched them against the custom ICMP rule.
- When matching packet was detected, Snort generated an alert.
- This demonstrated the basic working principle of a rule-based IDS:



```
sakshigakshi-VirtualBox:~/Desktop/snort3/build$ cat /usr/local/etc/snort/rules/local.rules
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:10000001; rev:1;)
sakshigakshi-VirtualBox:~/Desktop/snort3/build$ sudo sed -i 's/\$HOME_NET/\$HOME_NET/g' /usr/local/etc/snort/rules
/local.rules
sakshigakshi-VirtualBox:~/Desktop/snort3/build$ cat /usr/local/etc/snort/rules/local.rules
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:10000001; rev:1;)
sakshigakshi-VirtualBox:~/Desktop/snort3/build$ snort -c /usr/local/etc/snort/snort.lua -T
-----
o")-   Snort++ 3.12.2.0
-----
Loading /usr/local/etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  host_tracker
  hosts
  network
  packets
  process
  stream
  stream_ip
  stream_icmp
  stream_tcp
  stream_udp
  stream_user
  stream_file
  arp_spoof
  back_orifice
  dns
  netflow
  pop
  rpc_decode
  sip
  socks
  ssh
```

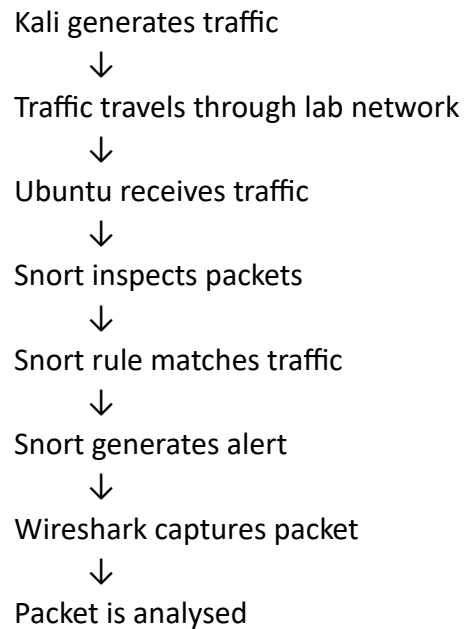
```
search engine (ac_bnf)
  instances: 2
  patterns: 438
  pattern chars: 2602
  num states: 1832
  num match states: 392
  memory scale: KB
  total memory: 71.2812
  pattern memory: 19.6484
  match list memory: 28.4375
  transition memory: 22.9453
appid: MaxRss diff: 2848
appid: patterns loaded: 300
-----
pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
** [0] enp0s3
09/20-14:13:29.776960 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.56.106 -> 192.168.5
6.105
09/20-14:13:29.778506 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.56.105 -> 192.168.5
6.106
09/20-14:13:30.779147 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.56.106 -> 192.168.5
6.105
09/20-14:13:30.779302 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.56.105 -> 192.168.5
6.106
09/20-14:13:31.780080 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.56.106 -> 192.168.5
6.105
09/20-14:13:31.780302 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.56.105 -> 192.168.5
6.106
09/20-14:13:32.781806 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.56.106 -> 192.168.5
6.105
09/20-14:13:32.781887 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.56.105 -> 192.168.5
6.106
```

1.10 PORT SCAN DETECTION

- Controlled network scanning was performed from Kali Linux to observe how Snort responds to suspicious scanning activity.
- Nmap was used to generate scanning traffic.
- Example:
Nmap <Ubuntu-IP>
- The scan generated multiple packets toward different ports.
- This traffic can be useful for testing IDS detection because port scanning is commonly associated with reconnaissance activity.
- Snort rules can be created to identify specific scanning patterns or suspicious connection attempts.
- The scan traffic was also examined using Wireshark to understand the packets generated during the test.

1.11 SNORT AND WIRESHARK CORRELATION

- ✓ Snort and Wireshark were used together during the investigation.
- ✓ The workflow was:



- ✓ This correlation provides a better understanding of how an IDS identifies network activity.
- ✓ Snort provides the detection/alert, while Wireshark provides detailed packet-level analysis.

1.12 IMPORTANT COMMANDS USED

- (1) Check Snort Version
`snort -V`
- (2) Test Snort Configuration
`snort -c /usr/local/etc/snort/snort.lua -T`
- (3) View routing information
`ip route`

- (4) Test connectivity
ping <IP-address>
- (5) Check network connections
netstat -an
- (6) Run Nmap scan from Kali
Nmap <target-IP>

1.13 TROUBLESHOOTING

Several issues were encountered during the project. Troubleshooting these problems provided practical experience with Linux networking, Snort configuration, and IDS testing.

15.1 Snort Package Not Found

- Initially, the Snort package could not be installed directly using the package manager.
- Solution:
Snort 3 was built from source after installing the required dependencies and libraries.

15.2 Configuration Validation Problems

- Configuration errors were encountered while setting up the Snort configuration and rules.
- Solution:
- The configuration was checked using:

```
snort -c /usr/local/etc/snort/snort.lua -T
```

- Configuration and rule errors were corrected until the validation completed successfully.

15.3 Network Connectivity Problems

- Initially, the Kali and Ubuntu machines were unable to communicate correctly.
- Solution
The VirtualBox network adapters, IP addresses, and routing configuration were checked.
Commands such as:

```
ip addr  
ip route  
ping <IP>
```

were used to identify and resolve connectivity problems.

15.4 ICMP Alert Not Appearing

- At one stage, ping traffic was generated but the expected Snort alert was not immediately visible.
- Solution:
The following were checked:

- Correct network interface
- IP addresses
- Snort configuration
- Local rule file
- Rule syntax
- Snort monitoring interface
- Network connectivity

- After correcting the configuration and ensuring that Snort was monitoring the correct interface, traffic detection could be tested again.

15.5 HTTP Traffic Detection Issue

- HTTP detection was also tested during the project, but the expected traffic was not consistently detected.
- Troubleshooting:
 1. The network connection, target service, Snort interface, rule configuration, and Wireshark capture were checked.
- This demonstrated an important IDS troubleshooting principle: **an alert cannot be generated if the relevant traffic does not reach the interface being monitored or does not match the configured rule.**

1.14 RESULTS

The project successfully demonstrated the basic operation of a Network Intrusion Detection System using Snort 3.

The following activities were performed:

- Snort 3 was installed and configured.
- Snort configuration was successfully validated.
- Custom detection rules were created.
- ICMP traffic was generated and tested.
- Network scanning traffic was generated using Kali Linux.
- Snort alerts were monitored.
- Network packets were captured using Wireshark.
- Snort alerts were correlated with packet-level traffic.
- Network and configuration issues were identified and resolved.

1.15 LEARNING OUTCOMES

This project provided practical knowledge of:

- Network Intrusion Detection Systems
- Snort 3 installation
- Snort configuration
- Custom rule creation
- ICMP traffic analysis
- Network scanning
- Nmap
- Wireshark packet analysis
- Linux networking
- Virtual machine networking
- Security alert investigation
- Troubleshooting IDS configurations
- Correlation between IDS alerts and packet captures

1.16 LIMITATIONS

- The project was implemented in a controlled virtual laboratory environment.
- The detection capabilities were limited to the rules and traffic patterns configured for testing. A production IDS environment would require a larger rule set, continuous monitoring, proper tuning, logging, alert management, and integration with other security technologies such as SIEM platforms.
- The project also focused primarily on demonstrating Snort's detection capabilities rather than deploying a complete enterprise-scale IDS infrastructure.

1.17 FUTURE SCOPE

The project can be extended by adding:

- SSH brute-force detection
- DNS monitoring
- HTTP attack detection
- Web attack detection
- SYN scan detection
- Suspicious TCP traffic detection
- More advanced custom Snort rules
- Automated alert logging
- SIEM integration
- Wazuh integration
- Real-time security dashboards
- Automated incident response

The project could also be expanded into an IDS/IPS laboratory by testing Snort in prevention mode.

1.18 CONCLUSION

- ❖ This project demonstrated the practical implementation of Snort 3 as a Network Intrusion Detection System in a virtual cybersecurity laboratory.
- ❖ Snort was successfully configured to inspect network traffic and generate alerts based on custom detection rules. Kali Linux was used to generate controlled network traffic, while Wireshark provided detailed packet-level analysis.
- ❖ The project helped develop practical skills in network monitoring, IDS rule creation, packet analysis, Linux networking, alert investigation, and cybersecurity troubleshooting.
- ❖ Overall, the project provided hands-on experience with the fundamental workflow of detecting and analyzing suspicious network activity using an IDS.